

Solution Requirements

Date	13 July 2026
Team ID	xxxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The current prototype does not implement user login. The ER diagram proposes a Users table (UserID, Name, Email, Password, Role) for email/password-based authentication in a future full version.	Medium
2	Authorization levels	Two roles are proposed via the Users table's Role field: Applicant (submits details, views own prediction) and Admin (manages the ML model, views all applicants and predictions).	Medium
3	External interfaces	The system does not call any third-party APIs. It loads a locally trained scikit-learn model (credit_card_model.pkl) via joblib directly	Low

		within the Flask app, with no external database currently connected.	
4	Transactions processing	The core transaction is a single prediction request: the user submits applicant details (17 features) via a web form; the Flask /predict route passes them to the trained Random Forest model and returns an Approved/Rejected result.	High
5	Reporting	The app currently shows a single prediction result page (result.html) per submission. The ER diagram's Approval_Prediction table proposes storing prediction history to support future reporting, such as approval-rate and risk-category summaries.	Low
6	Business rules, etc.	The model classifies applicants as approved or rejected based on patterns learned from historical application and credit payment data (e.g. income, employment length, family status, and overdue payment history).	High

7	Compliance to laws or regulations	This is an academic/portfolio project built on a public, anonymized credit application dataset. No formal regulatory compliance (e.g., GDPR, Fair Credit Reporting Act) has been implemented; a production deployment handling real applicant data would require a compliance review.	Low
8	<i>Other</i>	Basic input validation and error handling are implemented in the /predict route (try/except), returning an error message if required form fields are missing or invalid.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Since the model is a single lightweight Random Forest served locally (no external API calls), prediction requests should return quickly after form submission.	Prediction result returned in < 1 second

2	Scalability	The app is a single-model Flask service intended for demo/portfolio-scale use. Gunicorn (in requirements.txt) allows running multiple worker processes if concurrent load increases.	Confirm target based on deployment plan (e.g., handles X concurrent demo users)
3	Security & Data Privacy	Applicant data submitted includes sensitive personal and financial fields (income, employment, family status). The current prototype does not implement encryption or access control.	e.g., HTTPS in production; encryption at rest for any persisted applicant records
4	Reliability & Availability	As a small demo Flask app (not yet running behind a managed production environment), no formal uptime SLA is currently defined.	Confirm target based on hosting plan (e.g., 99% uptime during evaluation period)
5	Usability & Accessibility	The interface is a single labeled HTML form covering all 17 model inputs, with a plain-language result (Approved/Rejected) shown after submission.	Clear field labels; result understandable without technical knowledge
6	<i>Other</i>	Maintainability: the trained model is stored as a separate artifact (credit_card_model.pkl) that can be retrained or replaced without changing app.py.	